The Delegation Boundary: Using LLMs to Author Game Design Documents

Course: CSYE 7270 · Virtual Environments and Real-Time 3D · Take-Home Midterm
Category: A - AI Ideation: Design & Code Generation

Topic Claim

"After reading this piece, a practitioner will understand **using LLMs to generate and critique Game Design Documents** well enough to **decide which sections of a GDD to delegate to AI vs. protect from it** without making the mistake of **treating LLM-generated GDD content as design authority, producing documents that are internally consistent but mechanically unplayable.**"

- **X - Application:** LLMs as GDD co-author and critic
- **Y - Decision:** Deciding what to delegate to AI vs. what requires human design judgment
- **Z - Failure prevented:** The "coherent nonsense" GDD - reads well, but describes a game that can't be built or balanced

Section 1: The Scenario

It is 9:00 a.m. on a Monday. By Wednesday at 9:00 a.m., Mara Chen - lead designer at a four-person indie studio - must deliver a complete Game Design Document to a publisher's pitch committee. The game is *Ossuary*, a roguelike dungeon crawler in which the player inhabits the skeleton of a long-dead knight, reassembling their own body piece by piece from the bones of fallen enemies. The central mechanical conceit: each bone equipped is also a weapon, a stat modifier, and a consumable resource. Break your own femur across a boss's skull and you've dealt damage, triggered a bleed effect, and permanently sacrificed your kick attack until you find a replacement. The design is genuinely novel in the sense that matters here: no shipped game in the model's training corpus makes bones simultaneously a weapon, a stat modifier, and a consumable sacrifice - which means the model has no correct answer to draw from when the document demands one.

This is not a hypothetical. Pre-production crunch of this kind is endemic in indie development, where pitch cycles are externally imposed and teams are too small to parallelize documentation work. Mara has a vision - a mood board, a core loop sketched on a whiteboard, three pages of combat notes - but a GDD runs to forty or sixty pages when done properly: market positioning, player experience pillars, system diagrams, progression economy, enemy faction logic, UI wireframe descriptions, audio direction, and a build roadmap. She cannot write all of it from scratch before Wednesday.

She opens a conversation with an LLM.

What happens next depends entirely on how she uses it. Used well, the model functions as a tireless first-draft engine: it can populate boilerplate sections, propose enemy archetypes for her to evaluate, articulate genre conventions she already knows but hasn't yet transcribed, and surface internal contradictions in her notes before they calcify into the document. Used carelessly, it will generate a GDD that reads like a GDD - structured, confident, internally consistent - while quietly making design decisions she never sanctioned. The document will sound like someone decided. No one did.

The question this piece answers is not whether Mara should use the LLM. She should. The question is which parts she can delegate, which parts she must protect - and more importantly, why the line falls precisely where the novel mechanic begins to demand answers the training corpus cannot contain.

Section 2: The Mechanism

When Mara types her first prompt - something like "Generate the progression economy section of a GDD for a roguelike dungeon crawler with a body-modification mechanic" - she is not querying a database or invoking a rules engine. The model returns text that fits the pattern of what follows prompts like hers. More precisely: she is sampling from a probability distribution over text, shaped by everything in the training corpus that resembles game design documentation.

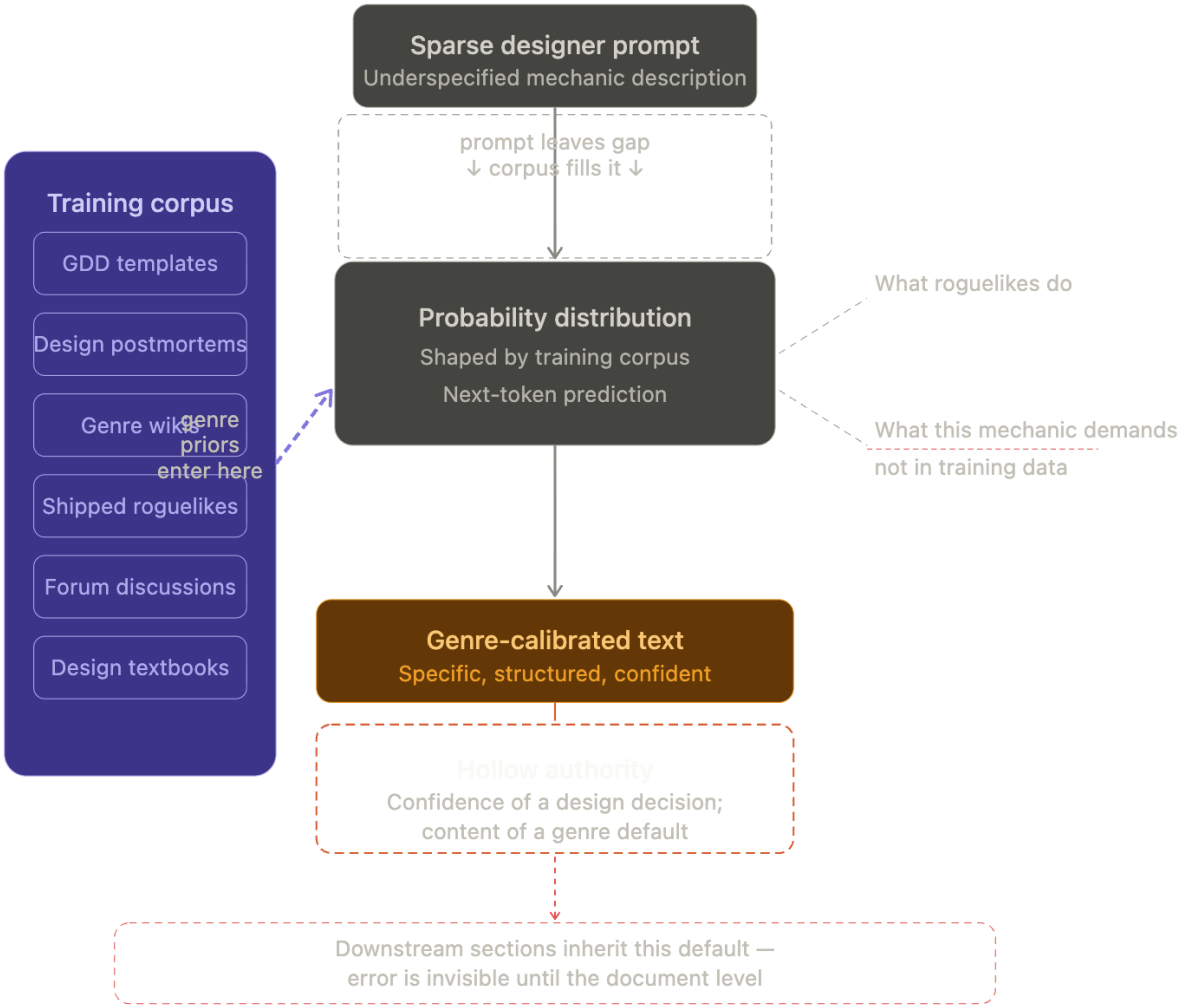
That corpus includes GDD templates, game design textbooks, postmortem essays, genre wikis, forum discussions, and shipped roguelike documentation. From this material the model learned, with high statistical confidence, what a progression economy section looks and sounds like: a leveling curve description, a resource scarcity model, a table of unlock gates, prose about player empowerment. It learned the vocabulary of design authority - words like "ensures," "creates," "rewards" - because published design documents use those words constantly. It returns text that deploys that vocabulary fluently, in the correct structure, at approximately the correct length.

What it did not learn is whether the bone-break mechanic Mara described in her prompt is internally coherent, or how a consumable-limb economy interacts with a run-length expectation, or whether sacrificing movement in exchange for burst damage collapses on floor three when enemy speed scales faster than bone availability. The model has no ground-truth model of *Ossuary* because *Ossuary* does not exist in its training data. It has only what Mara told it in the prompt, plus its statistical expectations about roguelikes in general.

This is the mechanism behind what we might call hollow authority: prose that carries the confidence of a design decision while encoding only a genre default. The model fills the gap between Mara's sparse prompt and a complete section by drawing on genre convention. A typical roguelike gates major abilities at run-hours one, three, and six, so the generated document will propose something similar - not because that cadence fits the bone-break loop, but because it fits the genre. The output will read as if someone made this decision deliberately. The confidence of the prose is not a signal of design validity; it is a property of next-token prediction.

The critique function works identically. Ask the model to "review this progression section for balance," and it will compare the text against genre expectations and flag deviations - which is useful only when genre convention is the correct standard, and precisely wrong when the mechanic's whole value is its deviation from that convention. The model critiques what it can measure. What it cannot measure, it does not flag. The section passes review and the error is invisible.

Understanding this mechanism is the prerequisite for using the tool correctly. It is not a limitation to route around; it is the operating condition the practitioner must design for.



Section 3: The Design Decision

The practitioner's first instinct is usually to treat delegation as a matter of time. Give the model the sections you haven't gotten to yet; write the sections you care about. This is the wrong frame. The correct frame is dependency: which sections derive their validity from the core mechanic, and which sections derive their validity from the genre.

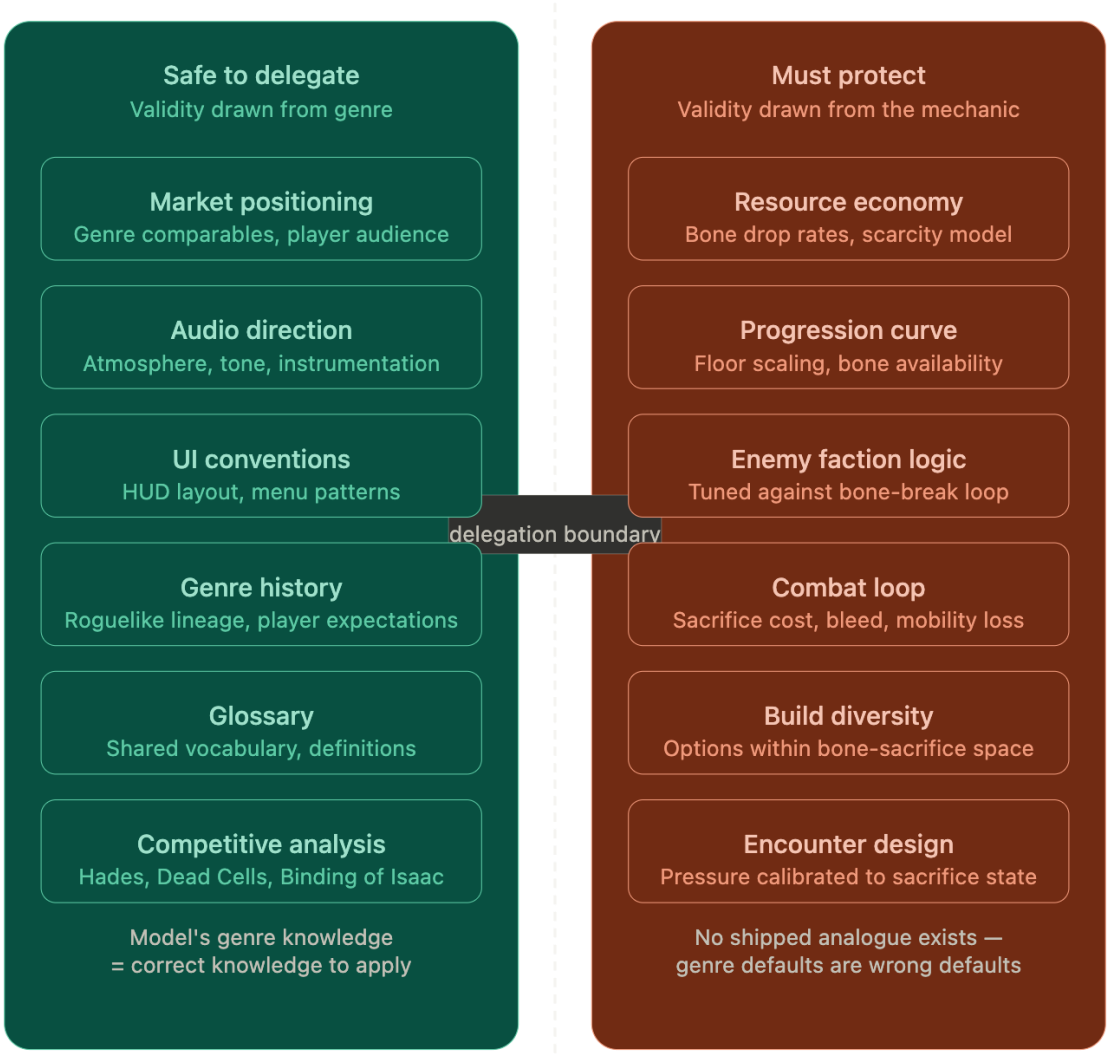
Sections that derive from the genre are safe to delegate. Because these sections appear in substantially identical form across dozens of shipped roguelikes in the training corpus, the model's statistical expectations are the correct expectations to apply. Market positioning, genre history, player experience pillars stated at a high level of abstraction, audio direction, UI convention descriptions, glossary definitions, and competitive analysis all belong here. When

Mara asks for a two-paragraph audio direction section - atmospheric, percussive, sparse - the model's genre knowledge is the correct knowledge to apply. She should read it, not rewrite it. By contrast, the bone-break economy has no training analogue - the model fills it with genre defaults, and those defaults are wrong.

Sections that derive from the core mechanic must be protected. In *Ossuary*, this means the resource economy, the progression curve, the enemy design logic, and the moment-to-moment combat loop. These sections do not have correct answers in the training data because the bone-break mechanic has never shipped. Any numbers the model proposes - bone drop rates, floor scaling coefficients, bleed damage ratios - are extrapolated from conventional roguelike tuning. They will produce a document that is internally consistent against genre norms and externally wrong against the actual design problem.

This distinction is architectural rather than preferential because errors in the mechanic-dependent sections propagate. A GDD is not a collection of independent chapters; it is a dependency graph. If the progression economy assumes a conventional resource scarcity model, the enemy faction section will be written to challenge a conventional resource flow. The encounter design will balance against that. The build diversity section will enumerate options that make sense within it. By the time Mara reads the full draft, the bone-break mechanic has been quietly replaced by a genre-average roguelike wearing *Ossuary*'s premise as a costume. Each section, reviewed in isolation, passes. The drift is only visible at the document level - by which point it has already propagated everywhere.

The delegation boundary must be drawn before the first prompt is written, not inferred from the output after the fact. Attempting to set the boundary after reading the draft fails because review is then calibrated against the generated text - the designer looks for internal inconsistency rather than comparing against the mechanic's actual demands, which she has not yet written down. Mara must know what her mechanic demands of every downstream section before she can know which sections the model is allowed to touch.



Section 4: The Failure Case

That detection failure is not abstract. Suppose Mara delegates the progression economy. She is short on time, the section is long, and she tells herself she will review the numbers before the pitch. She prompts: "Write the resource economy section for a roguelike dungeon crawler where the player collects and equips bones as weapons and stat modifiers." The model returns four pages. They are coherent, numerically specific, and structurally complete.

The model has, without flagging it, made a foundational assumption: bones are a persistent currency. In most roguelikes, the primary resource - gold, souls, relics - accumulates across a run

and is spent at discrete upgrade nodes. The model assigns bones to this conventional behavior because the prompt did not explicitly contradict it. The section describes a drop rate of 3-5 bones per enemy, a floor-end shop where bones are traded for upgrades, and a scaling curve tuned, so the player reaches a viable build by floor five.

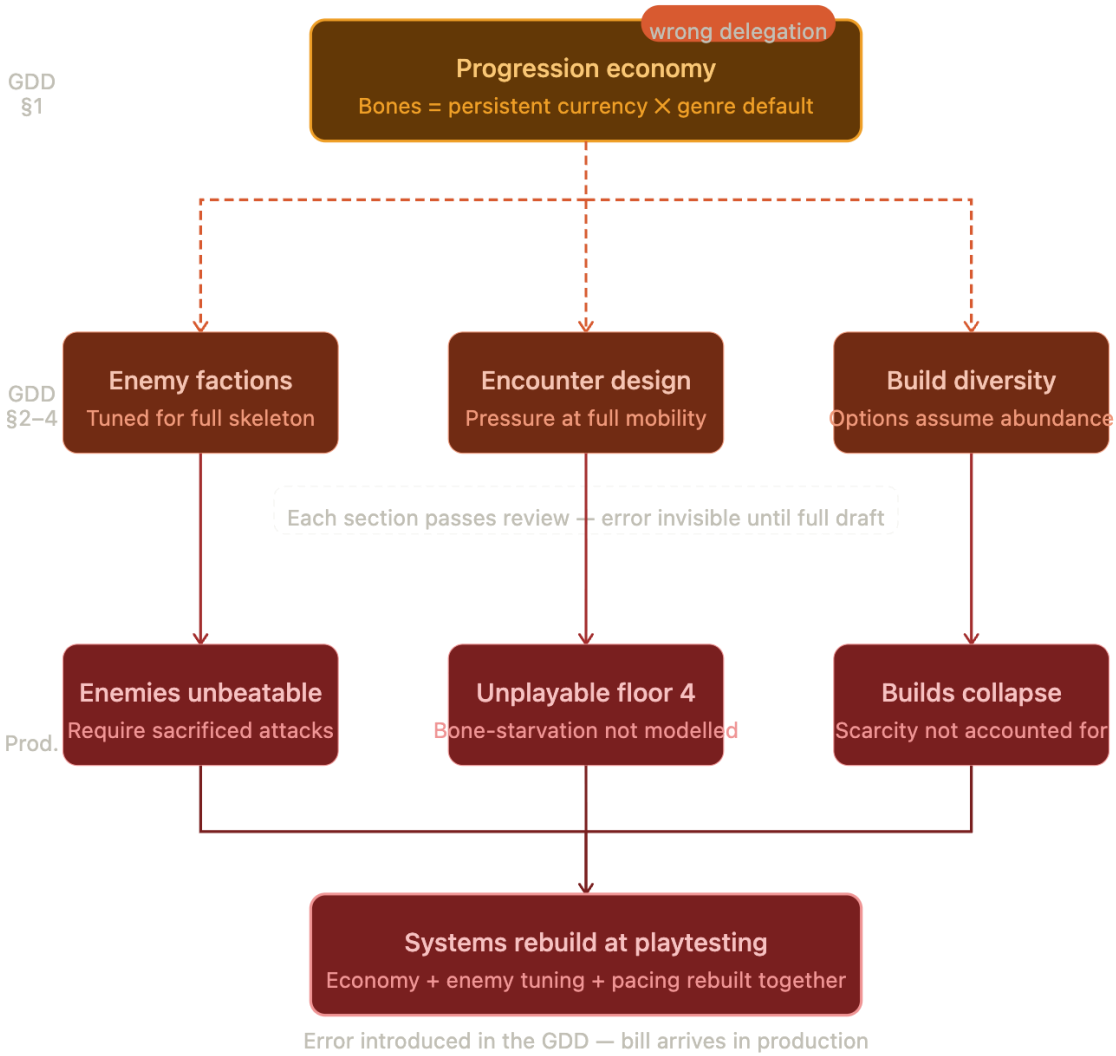
This is internally consistent. It is also the wrong game.

In *Ossuary*, bones are consumable. Breaking a femur is a tactical decision with a permanent in-run consequence - you lose the kick attack until a replacement drops. The economy is defined by scarcity and sacrifice, not accumulation and shopping. The model's document describes a system with abundant, fungible resources and predictable upgrade pacing. The moment the progression section is accepted, every downstream section is written against it.

The enemy faction section, generated next, designs encounter pressure around a player with a complete skeleton by floor three. Enemy speed and armor values are tuned to require full mobility at that stage of the run - kick attacks included. The encounter design is internally consistent with the economy. Both are wrong.

In production, a developer implementing the enemy faction spec will build enemies tuned to require kick attacks at floor three - the very attacks the player will have sacrificed to the bone-break mechanic - making the enemies unbeatable not despite the mechanic but because of it. The system will appear to function in isolation: the economy populates; the enemies spawn; the UI displays correctly. The unplayability will only surface in playtesting, at which point the cost to fix is not a document revision, but systems rebuild - economy, enemy tuning, and encounter pacing simultaneously. The error was introduced in the GDD; the bill arrives in production.

This is failure mode. The LLM did not hallucinate. It produced exactly what next-token prediction produces when given an underspecified prompt: a genre-calibrated answer to a question the designer did not know she was asking.



Section 5: The Exercise

Run the following two API calls in sequence. Use any model with a standard chat completion interface; the effect is not model-specific.

Call 1 - Underspecified prompt

- System prompt: "You are a game design document author. Be specific and numerically precise."

- User prompt: "Write the resource economy section for a roguelike dungeon crawler where the player collects and equips bones as weapons and stat modifiers. Include drop rates, upgrade nodes, and a floor-by-floor progression curve."

Save the output without editing it.

Call 2 - Constrained prompt

- System prompt: "You are a game design document author. Be specific and numerically precise."
- User prompt: "Write the same section, but bones are consumable - equipping a bone destroys it on use, and the player permanently loses that attack until a replacement drops. Redesign the economy accordingly."

Read both outputs side by side. The first will describe accumulation and shopping; the second will attempt scarcity and sacrifice. Now stress-test the second document with one question the model did not ask itself: *What happens on floor four if the player has broken every long-bone and no replacements have dropped?* The document will not answer this, because the model generated the economy from the center of the distribution - average drop rates, average run length, average player behavior. This is structural, not a prompt-writing failure: next-token prediction rewards plausible completions, and plausible completions assume average inputs. Floor-four bone-starvation is statistically rare in the corpus; it is common in the actual design problem. Write down exactly where the second document progression curve becomes unresolvable without designer intervention. That gap is the boundary the model cannot cross.

The argument above establishes where the boundary is and why. What it doesn't resolve is how much specificity is required to push that boundary - at what prompt density does the model's genre-filling stop being wrong? That's an empirical question. One approach: generate economy sections at progressively higher mechanic specificity, then stress-test each against the same tail case - bone-starvation on floor four - and measure where the model's output first requires no designer intervention to resolve it.

AI is a pipeline decision, not a magic layer. Where you put it - and how you constrain it - determines what your game becomes.